الملف الثامن: الدليل التشريحي المفصّل لكل وحدة برمجية في النظام (Low-Level Module Breakdown)

هذا الملف يُعتبر "دليل المطوّر" (Developer Guide). سنقوم فيه بتشريح كل مكوّن في المشروع، والدخول إلى مجلداته الداخلية، وشرح وظيفة كل حزمة (Package) برمجية، وكيف تعمل أجزاؤها من الداخل وتتواصل مع بعضها.

1. تشريح وكيل نظام ويندوز (win_edrAgent/internal)

الوكيل مبني بلغة Go، وهو مقسم إلى حزم (Packages) مستقلة تقوم بوظائف معزولة لضمان سهولة الصيانة والأمان.

حزمة `collectors/` (جامعات البيانات)

- **كيف تعمل؟** هذه الحزمة تحتوي على الحساسات (Sensors).
  - `etw.go` / `etw_cgo.c` : يستخدم كود C للوصول إلى `StartTrace` و `EnableTraceEx2` في نواة Windows. يقرأ الأحداث الأمنية في الوقت الفعلي (عمليات، ملفات، شبكة) ويحولها إلى هياكل بيانات Go.
  - `wmi.go` : يُنشئ جلسات `COM` للاتصال بـ Windows Management Instrumentation. يقوم بعمل استعلامات (WQL) دورية لجلب معلومات النظام العميقة (مثل البرامج المثبتة أو الأجهزة الطرفية).
  - `network.go` : يستدعي `IP Helper API` أو ينفذ أوامر لجلب جداول التوجيه وحالة اتصالات TCP الحية (ESTABLISHED/LISTENING).
  - `registry.go` : يُراقب مفاتيح السجل الحرجة (مثل `Run` , `RunOnce`) باستخدام `RegNotifyChangeKeyValue` .

حزمة `responder/` (محرك الاستجابة التلقائية)

- **كيف تعمل؟** تتلقى إشارات من `collectors` .
  - `engine.go` : يحتوي على طابور فحص (Scan Queue). عندما يُنشأ ملف جديد، ينتظر لـ 150 مللي ثانية (لضمان انتهاء الكتابة)، ثم يحسب له SHA-256.
  - `quarantine.go` : إذا طابق الملف توقيعاً خبيثاً، يقوم بتغيير صلاحيات الملف (ACLs) لمنع المستخدم من فتحه، وينقله إلى مجلد العزل المشفر `C:\ProgramData\EDR\quarantine` مع تغيير امتداده.

حزمة `command/` (معالج الأوامر)

- **كيف تعمل؟** هذه الحزمة مسؤولة عن تنفيذ الأوامر القادمة من الـ C2.

- handler.go : يستخدم نمط Goroutine Pool (حد أقصى مثلاً 8 أوامر متزامنة) حتى لا يستهلك أوامر الـ C2 معالج الجهاز (CPU).
- يعتمد على os/exec لتشغيل سكريبتات أو دوال داخلية. مثلاً أمر isolate_network ينفذ أوامر netsh advfirewall لمسح كل القواعد وإنشاء قاعدة واحدة تسمح بالاتصال بخادم الـ EDR فقط.

حزمة queue/ (طابور التخزين المحلي)

- **كيف تعمل؟** تمنع فقدان الأحداث عند انقطاع الإنترنت.
  - wal.go (Write-Ahead Log): تقوم بتسلسل الأحداث بـ Protobuf وتكتبها على القرص في ملفات متتابعة بحجم أقصى (مثلاً 50MB للملف).
  - processor.go : عندما يعود الاتصال، يقرأ هذه الملفات بالترتيب الزمني (FIFO) ويُرسلها للخادم، ثم يحذف الملفات التي تم إرسالها بنجاح.

حزمة grpc/ (الاتصال الشبكي)

- **كيف تعمل؟** مسؤولة عن القناة الآمنة.
  - client.go : يُهيئ اتصال gRPC باستخدام شهادة mTLS المُخزنة محلياً. يستخدم grpc.Dial مع خيارات Keepalive (إرسال Ping كل 30 ثانية لتجنب قطع الاتصال من جدران الحماية).
  - reconnect.go : يُنفذ خوارزمية Exponential Backoff . إذا فشل الاتصال ينتظر (1s, 2s, 4s... حتى 30s) ويستمر في المحاولة.

حزمة enrollment/ (التسجيل الذاتي)

- **كيف تعمل؟**
  - تُولّد مفتاح RSA الخاص بالجهاز.
  - تبني (Certificate Signing Request) CSR وتُرفق معه معلومات الجهاز (Hardware ID, OS).
  - ترسل الـ CSR عبر HTTP/REST العادي (لأنه لا يملك شهادة بعد) مع الـ Enrollment Token .
  - يستلم الشهادة ويحفظها بشكل آمن باستخدام حزمة /security .

حزمة security/ و protection/ (حماية الوكيل نفسه)

- **كيف تعمل؟** (Anti-Tampering).
  - تقوم بتعيين (Discretionary Access Control Lists) DACLs صارمة على مسار تثبيت الوكيل ومفاتيح الـ Registry الخاصة به، بحيث لا يملك صلاحية تعديلها أو حذفها سوى المستخدم SYSTEM فقط.

2. تشريح مدير الاتصالات (connection-manager/internal)

يعمل كحلقة الوصل المركزية ويتبع معمارية نظيفة (Clean Architecture).

حزمة `/handlers` (واجهات برمجة التطبيقات - API)

- **كيف تعمل؟** هي الطبقة المواجهة للشبكة.
  - `grpc_server.go` : يستمع على المنفذ 50051. يطبق `Interceptor` للتحقق من هوية الوكيل (يستخرج الـ `Agent ID` من الـ mTLS Certificate). يستقبل `Stream` مستمر من الأحداث.
  - `rest_api.go` : باستخدام إطار عمل `Gin-Gonic` أو `Echo` . يمتلك `Middleware` للتحقق من الـ `JWT Token` الخاص بلوحة التحكم. يمتلك مسارات مثل `/api/v1/agents` و `/api/v1/commands` .

حزمة `/service` (منطق الأعمال - Business Logic)

- **كيف تعمل؟** العقل المدبر لمدير الاتصالات.
  - `agent_service.go` : يراقب حالة الأجهزة. إذا لم يصل `Heartbeat` لمدة دقيقتين، يُحدث حالة الجهاز في قاعدة البيانات إلى `Offline` .
  - `auth_service.go` : يُدير دورة حياة مستخدمي الـ SOC. يستخدم `bcrypt` للتحقق من كلمات المرور، ويولد `JWT Tokens` . يمتلك آلية الحماية من القوة الغاشمة (Brute-force) عبر تتبع المحاولات الفاشلة.
  - `automation_service.go` : يقرأ التنبيهات من `Kafka` (موضوع Alerts)، يُقيّم شروط الـ Playbooks المخزنة، وإذا طابقت، يُصدر أوامر (Commands) تلقائياً عبر `command_service.go` .
  - `certificate_service.go` : تعمل كـ `Local CA` (سلطة شهادات محلية). تستلم طلبات الـ CSR من الوكلاء، توقعها بمفتاح الـ CA الخاص بالمنصة، وتصدر شهادة X.509 صالحة لسنة.

حزمة `/repository` و `/database` (الوصول للبيانات)

- **كيف تعمل؟** التفاعل مع `PostgreSQL` .
  - تستخدم مكتبات `GORM` أو (ORM) `sqlx` لكتابة استعلامات قاعدة البيانات بأمان تام ضد `SQL Injection` .
  - تنفذ عمليات `CRUD` لجداول: الأجهزة، الأوامر، التنبيهات، المستخدمين، وكتب اللعب (Playbooks).

حزمة `/cache` (التخزين المؤقت)

- **كيف تعمل؟** تتصل بـ `Redis` .
  - تخزن الـ `Tokens` المُلغاة (Blacklist).
  - تستخدم آلية `Pub/Sub` لإرسال إشعارات التنبيهات الجديدة (New Alerts) وحالات الأوامر (Command Results) إلى خوادم الـ WebSocket حتى تصل للوحة التحكم مباشرة.

3. تشريح محرك سيجما (sigma_engine_go/internal)

هذا المكون متخصص بـ "المعالجة التحليلية العالية" ويعمل كـ Stream Processor (معالج تدفق).

حزمة `infrastructure/` و `handlers/` (الاتصال الخارجي)

- **كيف تعمل؟**
  - تستهلك البيانات من `Kafka` عبر `events-raw` موضوع (Topic).
  - تُقدم `REST API` عبر `handlers/` يسمح للوحة التحكم بإدارة قواعد Sigma (إضافة/تعديل/تفعيل).

حزمة `application/` و `rules/` (نواة سيجما)

- **كيف تعمل؟**
  - `rule_parser.go` : يقرأ ملفات `YAML` ويقوم بتحليلها (Parsing) إلى هياكل بيانات Go (AST - Abstract Syntax Tree).
  - `indexer.go` : بدلاً من فحص آلاف القواعد لكل حدث، يبني "فهرس مقلوب" (Inverted Index) يعتمد على فئة الحدث (مثلاً: قواعد فئة الشبكة، قواعد فئة العمليات).

حزمة `domain/` (منطق الكشف)

- **كيف تعمل؟**
  - `evaluator.go` : يأخذ الحدث (Event) والقاعدة المرشحة، ويقوم بتنفيذ عوامل سيجما (`contains` , `startswith` , `regex`).
  - `confidence_scorer.go` : كل كشف ناجح يحصل على "درجة ثقة" مبنية على عدد الحقول المتطابقة.
  - `aggregation.go` : يمنع تكرار التنبيهات لنفس السلوك. يجمع التنبيهات المتشابهة من نفس الجهاز خلال نافذة زمنية قصيرة (مثلاً 5 ثوانٍ) ويصدرها كتنبيه واحد مجمع.

حزمة `analytics/` و `ml/` (الارتباط والذكاء)

- **كيف تعمل؟**
  - `correlation.go` : يتتبع سلاسل العمليات (Process Lineage). يربط أحداثاً تبدو عادية وحدها (مثل: وورد فتح ملف، ثم تم استدعاء PowerShell) لتكوين سيناريو هجوم كامل (Tactic).
  - `/ml` : تقوم ببناء Risk Score نهائي يعتمد على البيئة المحيطة بالجهاز وأهميته للمنظومة (Criticality Weight).

4. تشريح خدمة البناء (Agent Builder)

خدمة بسيطة ولكنها حرجة جداً في عملية النشر.

كيف تعمل من الداخل؟

- **`api.go` :** واجهة REST تستقبل طلب بناء وكيل جديد محتوياً على (IP الخادم، المنفذ، واسم المنظمة، وEnrollment Token).
- **`builder.go` :**
  i. يقوم بنسخ الكود المصدري النظيف للوكيل إلى مجلد مؤقت (Temp Directory).
  ii. يقوم بتعديل ملف `config/default.go` ديناميكياً ليضع فيه عنوان الـ Connection Manager والـ Token باستخدام أدوات المعالجة النصية لـ Go.
  iii. يستدعي المجمع `go build` الخاص باللغة.
  iv. يستخدم أوامر `os/exec` لتنفيذ `make windows` أو تشغيل `GOOS=windows go build ...` .
  v. يُعيد الملف الناتج كاستجابة (`application/octet-stream`) HTTP إلى المتصفح ليتم تنزيله.

5. تشريح لوحة التحكم (Dashboard - React/Vite)

الواجهة الأمامية مبنية بطريقة تقسيم المكونات (Component-Based).

مجلد `/src/pages`

- يحتوي على الملفات الرئيسية لكل شاشة (تم تفصيلها بالكامل في الملف السادس). كل ملف يمثل "Route" مستقلاً في `react-router-dom` .

مجلد `/src/components`

- **كيف يعمل؟** يحتوي على المكونات القابلة لإعادة الاستخدام (Reusable Components) لضمان عدم تكرار الكود:
  - `Table.tsx` : جدول ذكي يدعم الفرز، الفلترة، والترقيم (Pagination).
  - `Drawer.tsx` / `Modal.tsx` : النوافذ المنبثقة والجانبية لعرض تفاصيل التنبيهات والأجهزة.
  - `Charts.tsx` : تغليف لمكتبة `Recharts` لعرض الرسوم البيانية.
  - `StatusBadge.tsx` : مكوّن صغير يقرأ حالة الحدث (مثلاً Critical) ويعطيه لوناً (أحمر) وشكلاً معيناً.

مجلد `/src/api`

- **كيف يعمل؟** (طبقة الاتصال بالخادم)

- client.ts : يستخدم axios . يقوم بحقن الـ Token في Headers لكل طلب. يقوم باعتراض (Intercept) الأخطاء؛ فإذا عاد الخادم بخطأ 401 Unauthorized ، يقوم بمسح الجلسة وتوجيه المستخدم لصفحة الدخول.
- يحتوي على دوال مُغلفة (Wrappers) مثل agentsApi.list() ، ما يجعل كود الصفحات نظيفاً وخالياً من تعقيدات الـ HTTP.

مجلد /src/hooks

- **كيف يعمل؟** يحتوي على Custom Hooks مخصصة.
  - useWebSocket.ts : يُدير اتصال الـ WebSockets. يُعيد الاتصال تلقائياً إذا انقطع (Reconnection Logic)، ويمرر الأحداث الجديدة إلى TanStack Query لتحديث الشاشة فوراً دون الحاجة لإعادة التحميل.

6. تشريح البنية التحتية والمكونات الداعمة (Infrastructure)

docker-compose.yml (المايسترو)

- **كيف يعمل؟** هو ملف إعداد يُعرف جميع الخدمات (Services) لتعمل معاً في شبكة افتراضية واحدة (Docker Network).
  - يُنشئ حاوية postgres ويحدد لها Volume لحفظ البيانات حتى لو أُعيد تشغيل الحاوية.
  - يُنشئ zookeeper و kafka ويضبط إعدادات المنافذ للتواصل الداخلي.
  - يُنشئ redis مع كلمة مرور قوية.
  - يُشغل الحاويات البرمجية (Connection Manager, Sigma Engine) ويمرر لها متغيرات البيئة (Environment Variables) مثل DATABASE_URL و KAFKA_BROKERS .

مجلد /db/migrations

- **كيف يعمل؟**
  - يحتوي على ملفات SQL (مثل CREATE TABLE agents ...).
  - أداة ה- Migrations (المدمجة في Connection Manager) تقوم بقراءة هذه الملفات وتطبيقها تصاعدياً على PostgreSQL.
  - تسجل رقم آخر ملف تم تنفيذه في جدول schema_migrations ، لمنع تكرار التنفيذ عند إعادة تشغيل الخادم. هذا يضمن أن قاعدة البيانات دائماً بنفس بنية الكود.

خدمة Nginx (البوابة)

- **كيف تعمل؟**
  - حاوية Nginx تعمل كـ Reverse Proxy.
  - nginx.conf : ملف الإعداد الذي يحدد قواعد التوجيه:

- أي طلب `HTTP GET /` يعيد ملف `index.html` الخاص بـ React.
- أي طلب لـ `api/v1/sigma/` يُمرر إلى حاوية `sigma-engine` على منفذها الداخلي.
- أي طلب لـ `/api/v1/` يُمرر لـ حاوية `connection-manager` .

- هذا يحمي الخدمات الداخلية؛ فهي لا تستمع على الإنترنت مباشرة، بل Nginx هو الوحيد المكشوف للعالم الخارجي.